Department of CSE(Artificial Intelligence & Machine Learning)

LAB MANUAL

Cloud Computing
(23UAMPCL3509)
(2023 PATTERN)

Class: T. Y. Btech
Term: I

INSTITUTE VISION AND MISSION

VISION

To achieve excellent standards of quality education by keeping pace with rapidly changing technologies and create technical manpower of global standards with capabilities of accepting new challenges.

MISSION

Our efforts are dedicated to impart quality and value-based education to raise the satisfaction level of all stake- holders. Our strength is directed to create competent professionals. Our endeavor is to provide all possible support to promote research and development activities.

DEPARTMENT VISION AND MISSION

VISION

To develop globally competent professionals with a strong foundation in Artificial Intelligence and Machine Learning, empowering them to drive innovation and positively impact industry and society.

MISSION

The department strives to:

M1: Provide a comprehensive, cutting-edge curriculum that keeps pace with the rapid advancements in AI and ML technologies by collaborating with leading companies.

M2 Nurture a diverse, globally-minded student community by providing practical, experiential learning opportunities for students to apply their knowledge and skills to solve real-world problems.

M3: Create a culture that promotes continuous learning, research, and entrepreneurial skills in order to give our students a lifelong passion for staying abreast of new developments..

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEOs):

PEO-I: To analyse, design and develop cost effective solutions to the real life problems by applying the acquired knowledge.

PEO-II: Adaptability to learn latest technological advancement and interdisciplinary approaches by engaging in a lifelong learning process.

PEO-III: Willingness to pursue higher education, entrepreneurship and research in the field of Artificial Intelligence.

PEO-IV: Being responsible towards society, environment, and ethical responsible team members with interpersonal and leadership skills.

Program Specific Outcomes (PSOs):

The graduates of the department will attain ability to:

PSO1: Apply the concepts and practical knowledge in analysis, design and development of sustainable solutions.

PSO2: Use problem-solving skills and develop multidisciplinary projects with advanced technologies and tools.

PSO3: Enhance the abilities to qualify for employment, higher studies and research in AI with ethical values.

G H Raisoni College of Engineering & Management, Pune
Second Year of CSE(Artificial Intelligence & Machine Learning) (2023 Pattern)
23UAMPCL3509: Cloud Computing

Teaching Scheme: **PR: 02 Hours/Week**	**Credit** **01**	**Examination Scheme:** **Ext: 25 Marks** **Total: 25 Marks**

Sr. No.	**List of Laboratory Assignments.**
1	Study and implementation of Infrastructure as a Service-Amazon Elastic Compute Cloud (EC2) using AWS Management Console .
2	Create AWS VPC For Add or remove an IPv6 CIDR block from your subnet.
3	Study & Implement Windows VM using the Terraform code.
4	Understand & Implement how hypervisors such as QEMU utilize the Linux KVM API to provide efficient hardware-assisted virtualization.
5	Configuring Athena to access data in Amazon S3 for Running Queries in AWS Athena.
6	Study the Migration tools and services to facilitate the migration of workloads to its cloud platform (use any cloud platform).
7	Study and Implement CloudFormation templates to provide the requested resources in your AWS account.
8	Implement container management with Kubernetes.
9	Implement common scenarios by using the AWS SDK for Kotlin with IAM.
10	Demonstrate the use of Cloud Computing Security Tools(Notable Open Source).

Course Objectives:

1. To understand the basic concepts of cloud computing.
2. To explain the concept and types of virtualization.
3. Understand & Design Microservices Architecture for Cloud.
4. Understand the role of cloud orchestration in modern IT infrastructures.

Course Outcome:

After completion of this course students will be able to:

CO1: Understanding cloud computing concepts, architecture and services.

CO2: Develop migration strategies from monolithic to microservice architecture and use tools like CoreOS, Amazon Sagemaker.

CO3: Analyze cloud platform architecture and services using AWS / GCP / Microsoft Azure.

CO4: Build secure, manageable and resilient cloud environments.

CO and PO Mapping with practical

Sr. No	Experiment List	Mapping With CO	Mapping With PO & PSO
1	Study and implementation of Infrastructure as a Service-Amazon Elastic Compute Cloud (EC2) using AWS Management Console .	CO1	PO1, PO3, PO5, PO9, PO10, PO12, PSO3, PSO2
2	Create AWS VPC For Add or remove an IPv6 CIDR block from your subnet.	CO1	PO1, PO3, PO5, PO9, PO10, PO12, PSO3, PSO2
3	Study & Implement Windows VM using the Terraform code.	CO2	PO1, PO2, PO3, PO4, PO5, PO9, PO10, PO12, PSO1, PSO2
4	Understand & Implement how hypervisors such as QEMU utilize the Linux KVM API to provide efficient hardware-assisted virtualization.	CO3	PO1, PO2, PO3, PO4, PO5, PO9, PO10, PO12, PSO1, PSO2
5	Configuring Athena to access data in Amazon S3 for Running Queries in AWS Athena.	CO3	PO1, PO2, PO3, PO5, PO9, PO10, PO12, PSO1, PSO2
6	Study the Migration tools and services to facilitate the migration of workloads to its cloud platform (use any cloud platform).	CO2	PO1, PO2, PO3, PO5, PO6, PO9, PO10, PO12, PSO1, PSO2
7	Study and Implement CloudFormation templates to provide the requested resources in your AWS account.	CO4	PO1, PO2, PO3, PO4, PO5, PO9, PO10, PO11, PO12, PSO1, PSO2
8	Implement container management with Kubernetes.	CO4	PO1, PO2, PO3, PO5, PO9, PO10, PO12, PSO1, PSO2
9	Implement common scenarios by using the AWS SDK for Kotlin with IAM.	CO4	PO1, PO2, PO3, PO4, PO5, PO8, PO9, PO10,

			PO12, PSO1, PSO2
10	Demonstrate the use of Cloud Computing Security Tools(Notable Open Source).	CO4	PO1, PO2, PO3, PO4, PO5, PO6, PO8, PO9, PO10, PO12, PSO1, PSO2
Content Beyond Syllabus			
10	DevOps Tooling by AWS to Adopt a DevOps Model.	CO2, CO3	PO1, PO2, PO3, PO5, PO7, PO9, PO10, PO11, PO12, PSO1, PSO2

CO Wise Weightage and Practical Marks

Marks of Each Experiment: 10

Total Marks of 10 Experiments: 100

Total Practical Marks: 25

Expt. No.	**Mapped CO**	**Marks of**			
		CO1	CO2	CO3	CO4
01	CO1	4			
02	CO2		3		
03	CO2		3		
04	CO3			2	
05	CO3			2.5	
06	CO3			2.5	
07	CO4				2
08	CO4				2
09	CO4				2
10	CO4				2
Weightage out of 100		10	20	30	40
Weightage out of 25		***4***	***6***	***7***	***8***

Rubrics for Practical Evaluation

Each experiment will be evaluated as per below rubrics:

Sr. No.	**Criteria**	**Poor** Does not meet expectation	**Satisfactory** Meet a few expectation	**Good** Meet expectation
1	Understanding of experiment **(0/1/2)**	Partial understanding of the problem statement and basic software concepts. Unable to explain logic or approach clearly.	Understands the problem statement and basic software concepts but needs guidance to implement the solution.	Clear understanding of problem statements, software concepts, and logic. Can implement the solution independently.
2	Program Design & Implementation **(0/1/2)**	Program structure is unclear, inefficient, or incorrect. Code contains major logical or syntactical errors.	Program structure is acceptable but needs improvement in logic, modularity, or efficiency. Minor errors present.	Well-structured, efficient, and logically correct program using appropriate constructs and modular design.
3	Output, Testing & Result Analysis **(1/2/3)**	Output is incorrect or incomplete. No proper testing or explanation of results.	Output is partially correct. Basic testing done with limited explanation of results.	Correct output for all test cases. Proper testing performed and results are clearly analyzed and explained.
4	Viva Voce and Submission**(1/2/3)**	Few questions answered without clear understanding. Assignment submitted late or incomplete.	Some questions answered correctly with basic understanding. Assignment submitted on time.	Most questions answered correctly with clear understanding. Assignment submitted on time and complete in all aspects.

Assignment No. 1

Aim: Study and implementation of infrastructure as a Service Amazon Elastic Compute Cloud (EC2) using AWS Management Console.

Software/libraries required:

- Web Browser (Chrome/Firefox/Edge)
- Active Internet Connection
- AWS Account (Free Tier eligible)
- AWS Management Console access
- (Optional for connecting to instance) PuTTY / SSH client for Windows, or built-in terminal for Linux/Mac
- Key Pair file (.pem / .ppk) for secure login

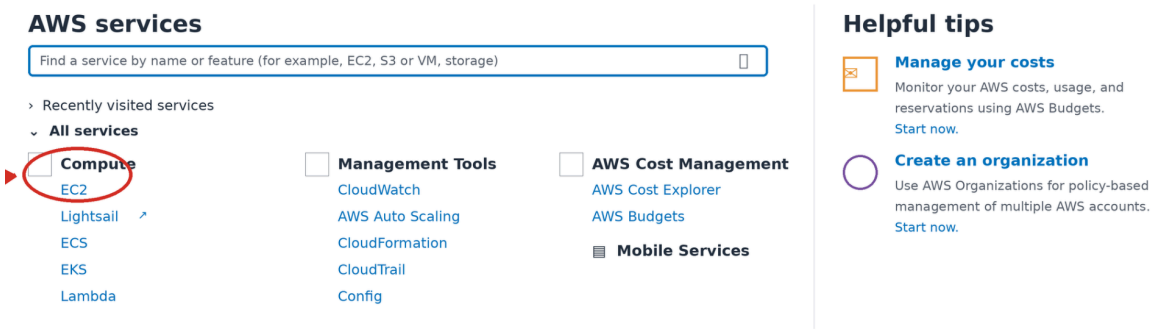
Theory:

Amazon EC2 (Elastic Compute Cloud) is an on-demand cloud computing service from AWS that lets you rent virtual servers to run applications. Instead of maintaining physical hardware, you can launch virtual machines with customizable memory, CPU, and storage.

Follow the below steps to create an EC2 instance in AWS (Amazon):

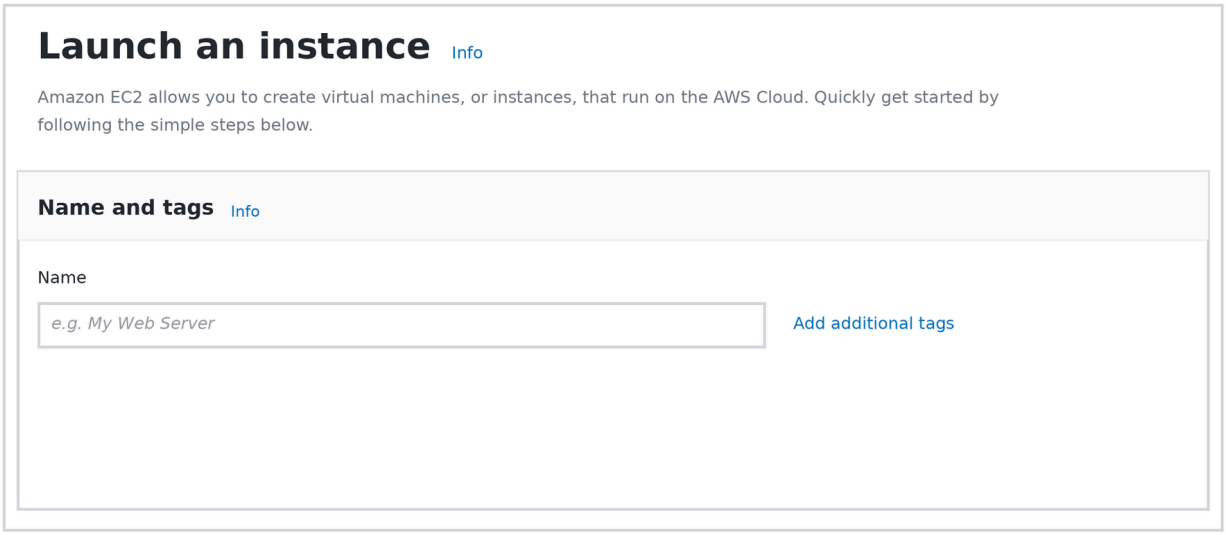
Step 1: Log in and Open EC2 Dashboard

- Sign in to your AWS account.
- Click Services > EC2.
- Check if any instances are running under Instances running.



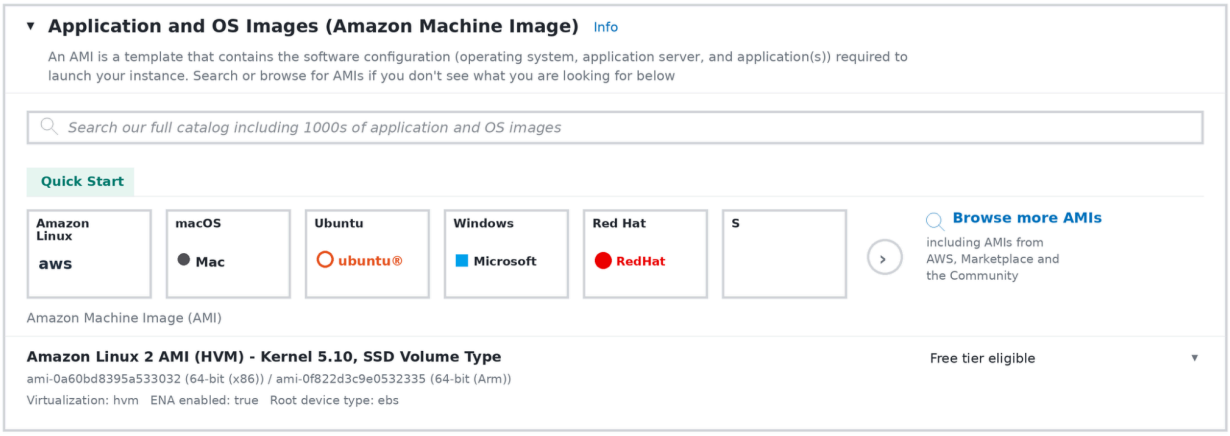
Step 2: Launch a New EC2 Instance

- Click Launch Instance.
- Enter a name for your instance.



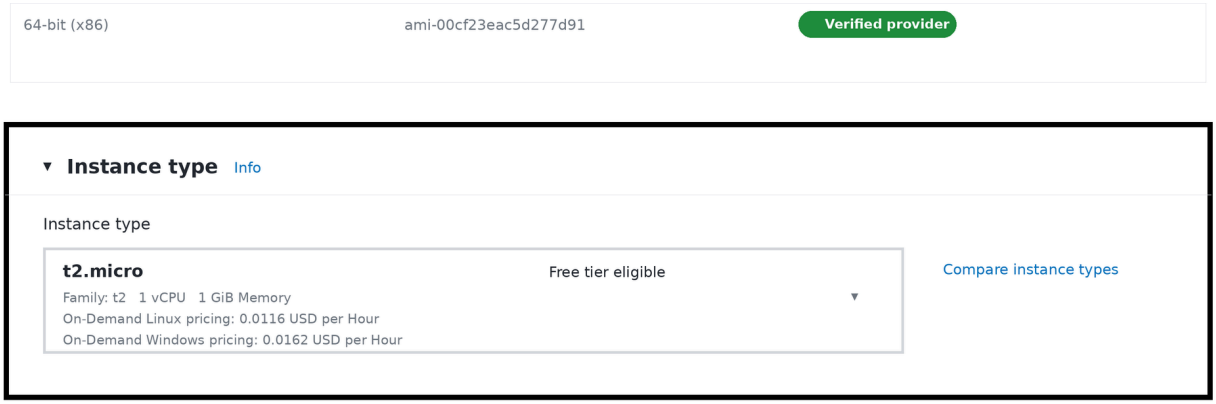
Step 3: Select an Amazon Machine Image (AMI)

- Select AMI - Required operating system from the available. There are different types of OS available, select the OS as per your requirement.



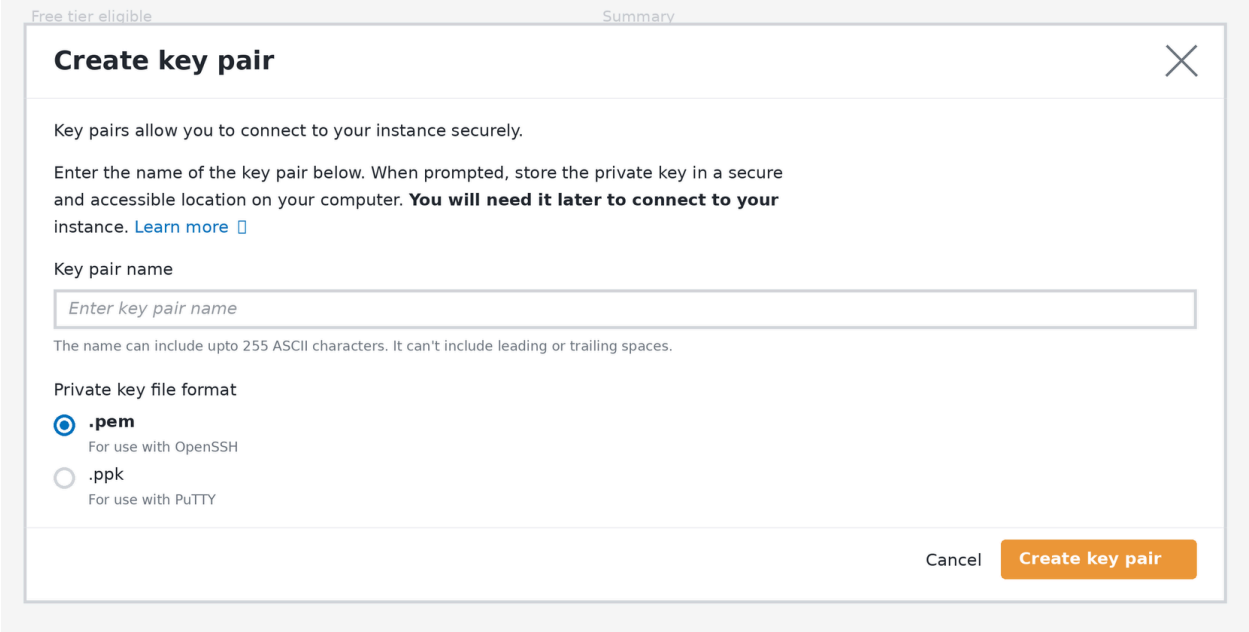
Step 4: Choose Instance Type

- By default, it selects a free tier of storage. (If you are eligible for the free tier). From the available storage specifications, select a free tier-eligible storage service.



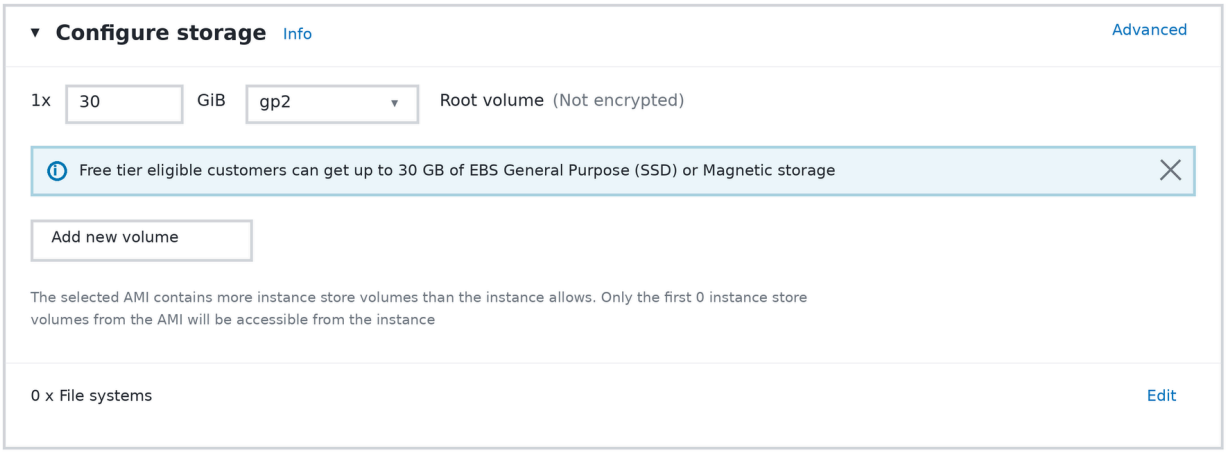
Step 5: Configure Key Pair

- Create a new key pair (select .pem format) and download it. • Keep this file safe—you'll need it to connect.



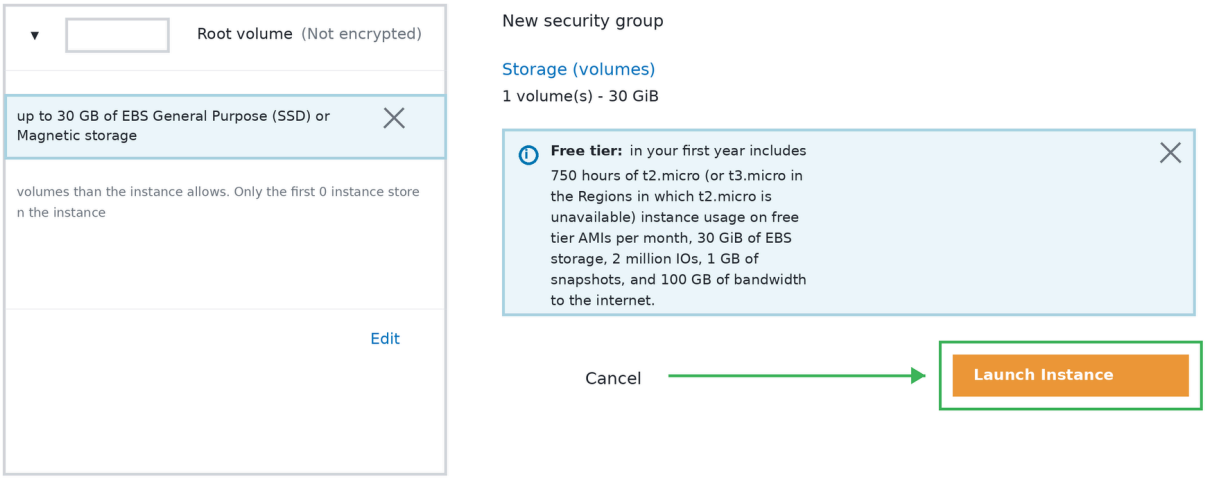
Step 6: Configure Network and Storage

- Use default network settings (VPC,subnet).
- Storage defaults to 8 GB (free tier eligible); keep it as is or adjust if needed.



Step 7: Review and Launch

- Verify all selections (ensure free tier eligibility if applicable). Click Launch to start your instance.



Oral Questions:

1. What is IaaS? How does it differ from PaaS and SaaS?
2. What is Amazon EC2, and what does "elastic" mean in this context?
3. What is an AMI? Name a few AMIs available on AWS.
4. What is the purpose of a key pair? What happens if you lose the private key?
5. What is a Security Group, and how is it different from a Network ACL?

Expected Output:

1. EC2 instance successfully launched with selected AMI (e.g., Amazon Linux/Ubuntu) and

instance type (e.g., t2.micro).
2. Key pair created/downloaded for SSH access; security group configured with required inbound rules (SSH port 22, HTTP port 80).
3. Instance state shows "Running" with public IPv4 address/DNS assigned.
4. Successful SSH login into instance from local terminal/PuTTY using key pair.
5. Screenshot of running instance details (Instance ID, public IP, status checks passed).

Conclusion:

Assignment No. 2

Aim: Create AWS VPC — Add or Remove an IPv6 CIDR Block from a Subnet

Software/libraries required:

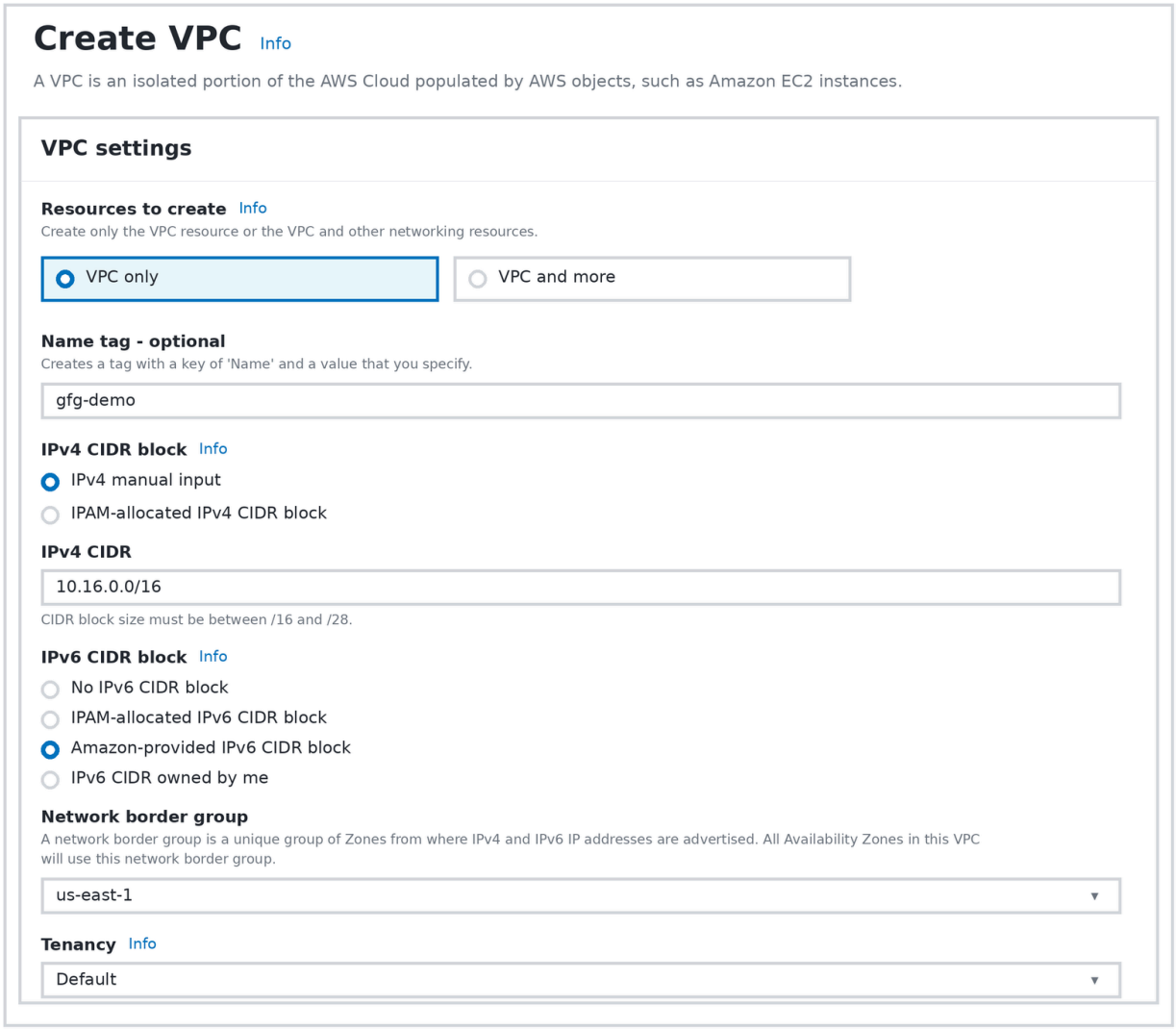
- An active AWS account with console access.
- Basic understanding of cloud computing concepts.
- Access to a computer with internet connectivity and SSH client (for connecting to Linux instances).
- Existing VPC with an IPv4 CIDR block.
- VPC must have an associated IPv6 CIDR block (Amazon provided or BYO).
- At least one subnet in the VPC.

Theory:

Amazon Web Service allows users to securely host applications and scale to millions of users without worrying about infrastructure provision and maintenance.

A Virtual Private Cloud (VPC) is a customizable, isolated network environment for deploying cloud resources. A default VPC is configured and ready for you to use when you create your account. You can also create your Custom VPC.

1. **Create or select your VPC**
   - Go to the VPC Dashboard in the AWS Console.
   - If your VPC doesn't have an IPv6 CIDR block yet, you can assign one there by choosing Actions > Edit CIDRs > Add IPv6 CIDR.



2. Create or select your Subnet

- In the VPC console, click on Subnets.
- When creating a new subnet, you'll see an option to assign an IPv6 CIDR block. It will let you pick a /64 subnet from your VPC's IPv6 CIDR block**.**

3. Add an IPv6 CIDR block to an existing subnet

- Select the subnet in the console.
- Choose Actions > Edit IPv6 CIDRs or similar options (depends on UI updates).
- You can then associate (add) a new IPv6 CIDR block to the subnet.

4. Remove an IPv6 CIDR block from a subnet

- In the subnet details, look for the IPv6 CIDR associations.
- Select the IPv6 CIDR block you want to remove and disassociate it.

Expected Output:

1. IPv6 CIDR block successfully associated with VPC (visible under VPC → CIDRs).
2. Subnet shows IPv6 CIDR block with state "Associated."
3. On removal, CIDR block shows state "Disassociated" and is removed from subnet.
4. Screenshot of before/after CIDR association status in AWS console.

Oral Questions:

1. What is a VPC, and why is it used?
2. What is CIDR notation? Give an example.
3. What is the difference between IPv4 and IPv6 addressing in AWS?
4. Can a subnet have only an IPv6 CIDR block without IPv4?

Conclusion:

Assignment No. 3

Aim: Study & Implement Windows VM using Terraform Code

Software/libraries required:

- An active AWS account with access credentials.
- Terraform installed and configured on the system.
- Basic understanding of cloud computing and virtualization.
- A computer with internet access.
- AWS key pair and a valid AMI ID for Windows OS.
- Familiarity with terminal/command line is helpful.

Theory:
Terraform is an open-source Infrastructure-as-Code (IaC) tool developed by HashiCorp. It allows users to define, deploy, and manage cloud infrastructure using a declarative configuration language called HCL (HashiCorp Configuration Language).

With Terraform, users can automate the provisioning of cloud resources such as virtual machines, networks, and storage across multiple providers like AWS, Azure, and GCP.
In this experiment, we use Terraform to provision a Windows EC2 instance on AWS Cloud, eliminating the need to manually configure infrastructure through the AWS Console.

Terraform Workflow:

Steps to Launch Windows VM on AWS using Terraform:

1. Step 1: Install and Initialize Terraform

- Download and install Terraform from the official site.
- Open terminal and run `terraform -version` to verify installation.

2. Step 2: Configure Provider and Resources

- Create a working directory and inside it, create a file named `main.tf`.
- Define the AWS provider and EC2 instance resource as shown below:
  Terraform code:

```
provider "aws" {
 region = "us-east-1"
```

```
}

resource "aws_instance" "windows_vm" {
 ami = "ami-0b2f6494ff0b07a0e" # Example AMI for  Windows Server
 instance_type = "t2.micro"
 key_name = "your-key-name"

 tags = {
 Name = "Terraform-Windows-VM"
 }
}
```

3. Step 3: Initialize and Plan Deployment

- Navigate to your Terraform directory in the terminal.
- Run the following commands:

```
terraform init
terraform plan
```

4. Step 4: Apply Configuration

- Run `terraform apply` and confirm when prompted.
- Terraform will now provision the instance.

5. Step 5: Verify the Instance

- Open AWS Console > EC2 Dashboard.
- Check that your Windows instance is running.

Expected Output:

A Windows EC2 instance should be created and visible in the AWS Console. You can connect to the instance using RDP (Remote Desktop Protocol) if properly configured.

Attach screenshot of the running instance here.

Oral Questions:

1. What is Infrastructure as Code (IaC)? Name other IaC tools besides Terraform.
2. What is the purpose of terraform init?
3. What is a Terraform provider?
4. What does the state file store, and why is it important?
5. Difference between terraform plan and terraform apply?

Conclusion:

Assignment No. 4

Aim: Understand & Implement How Hypervisors such as QEMU Utilize the Linux KVM API for Hardware-Assisted Virtualization

Software/libraries required:

- A system with a CPU that supports virtualization (Intel VT-x or AMD-V enabled in BIOS/UEFI).
- A Linux OS (preferably Ubuntu/Debian).
- Terminal access with sudo privileges.
- Installed tools: qemu-kvm, virt-manager, libvirt-daemon, bridge utils.
- ISO image of an operating system (like Ubuntu) for testing VM creation.

Theory:

Virtualization is a technique where a single physical machine is partitioned into multiple virtual environments, known as Virtual
Machines (VMs). This allows better utilization of hardware resources and isolation between applications.

Hypervisors:
Hypervisors are software or firmware that create and run VMs. They act as an interface between virtual machines and the physical hardware.

Type	Description	Examples
Type 1	Bare-metal; runs directly on hardware	VMware ESXi, Xen
Type 2	Hosted; runs on top of a host OS	VirtualBox, QEMU with KVM

1. **What is KVM?**

KVM (Kernel-based Virtual Machine) is a full virtualization solution built into the Linux kernel. It converts the Linux kernel into a Type 1 hypervisor by exposing virtualization extensions (Intel VT-x or AMD-V) to user-space applications like QEMU.

2. **What is QEMU?**

QEMU is a generic, open-source machine emulator and virtualizer. When run in emulation mode (without KVM), it simulates all hardware which is slower. With KVM enabled, QEMU becomes a fast virtualizer by utilizing CPU instructions and hardware features via the KVM

API.

3. **Relationship Between QEMU and KVM:**

QEMU acts as the frontend (user-space application) while KVM acts as the backend (kernel module). QEMU uses the KVM API to access privileged CPU instructions, page tables, and I/O access via the /dev/kvm device file.

Steps to Implement:

- **Step 1: Check Virtualization Support in CPU:**

egrep -c '(vmx|svm)' /proc/cpuinfo

- **Step 2: Install KVM and QEMU-related Packages:**

sudo apt update
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virt-manager –y

- **Step 3: Verify KVM is Enabled:**

lsmod | grep kvm

- **Step 4: Check Permissions and Add User:**

groups $USER
sudo usermod -aG libvirt $USER
sudo usermod -aG kvm $USER
reboot

- **Step 5: Launch GUI-based Virtual Machine:**

virt-manager
Step 6: Launch VM via QEMU Command Line:
qemu-system-x86_64 -enable-kvm -m 2048 -hda ubuntu.img - cdrom ubuntu.iso -boot d

Expected Output:

- Virtualization modules (kvm, kvm_intel or kvm_amd) loaded successfully.
- QEMU creates a VM with near-native performance using KVM.
- Virtual machine boots and installs guest OS successfully.
- Verification via virt-manager shows VMs using KVM backend.

Oral Questions:

1. What is the difference between Type 1 and Type 2 hypervisors?
2. What is KVM, and how does it turn Linux into a hypervisor?

3. What is the role of QEMU when paired with KVM?
4. What is hardware-assisted virtualization (VT-x/AMD-V)?
5. What happens if you run QEMU without the -enable-kvm flag?

Conclusion:

Assignment No. 5

Aim: To configure Amazon Athena to access data stored in Amazon S3 and execute SQL queries directly on that data using the Athena query engine.

Software/libraries required:

- An AWS account with access to S3 and Athena services.
- An existing S3 bucket with structured data (e.g., CSV, JSON, Parquet).
- IAM user with AmazonAthenaFullAccess and AmazonS3ReadOnlyAccess policies.
- Region set where Athena and the S3 bucket are both available.
- Optional: A simple dataset in CSV format uploaded to an S3 path.

Theory:

Amazon Athena is a serverless, interactive query service that enables users to analyze data directly in Amazon S3 using standard SQL. It reads files directly from S3, supports multiple data formats, and does not require provisioning infrastructure. Athena uses the Presto query engine under the hood.

How Athena Works:

- Athena reads files directly from S3.
- Supports CSV, JSON, ORC, Avro, and Parquet formats.
- Schema and tables are defined using the Data Catalog.
- Athena stores query results in an S3 output location.
- No infrastructure provisioning required.

Steps to Perform:

- **Step 1: Prepare Data in S3**

Upload a sample CSV (e.g., students.csv) to an S3 bucket: s3://your-bucket-name/data/students.csv

Example content:

id,name,dept,marks

1,Alice,CS,85

2,Bob,IT,78

- **Step 2: Create an S3 Output Folder for Athena**

In the same or another S3 bucket, create a folder like: s3://your-bucket-name/athena-results/

- **Step 3: Open Athena Console**
  Go to AWS Console > Athena.

In Settings (top right), configure the result location to: s3://your-bucket-name/athena-results/

- **Step 4: Create Database and Table**

Run the following SQL commands in the Athena query editor: CREATE DATABASE IF NOT EXISTS student_data;

```
CREATE EXTERNAL TABLE IF NOT EXISTS  student_data.students (
 id INT,
 name STRING,
 dept STRING,
 marks INT
)
ROW FORMAT SERDE
'org.apache.hadoop.hive.serde2.OpenCSVSerde' WITH SERDEPROPERTIES (
 "separatorChar" = ",",
 "quoteChar" = "\""
)
LOCATION 's3://your-bucket-name/data/';
```

- **Step 5: Run Sample Queries**

```
SELECT * FROM students;

SELECT * FROM students WHERE marks > 80;

SELECT dept, AVG(marks) AS avg_marks FROM students  GROUP BY dept;
```

Expected Output:

- Athena successfully queries the S3 file and returns tabular results.
- The query result files are stored in the specified S3 output folder.
- SQL-based queries on large datasets can be performed without provisioning a database engine.

Oral Questions:

1. What is Amazon Athena, and what problem does it solve?
2. Is Athena serverless? What does that mean for the user?
3. How does Athena pricing work?
4. What is the AWS Glue Data Catalog, and how does Athena use it?
5. What file formats does Athena support for querying?

Conclusion:

Assignment No. 6

Aim: To study various migration tools and services offered by cloud platforms such as AWS, Azure, or GCP, which facilitate smooth migration of workloads (data, applications, VMs, databases) to the cloud.

Software/libraries required:

- Basic understanding of cloud computing and virtualization.
- Internet-connected system with access to at least one cloud provider's console (e.g., AWS, Azure, GCP).
- Optional: Admin-level access or free-tier account on the selected platform.
- Sample workload (VM, DB, file server) to understand migration flow.

Theory:

Cloud Migration refers to the process of moving digital business operations — such as applications, workloads, data, and services — to a cloud computing environment. Cloud platforms provide dedicated Migration Services to simplify, accelerate, and secure this process.

Key Migration Scenarios:

Workload Type	Target	Example Tool/Service
VMs/Servers	Cloud VM	AWS MGN, Azure Migrate
Databases	Managed DB	AWS DMS, Azure Database Migration Service
Files/Storage	Cloud Object Store	AWS Snowball, Azure Data Box
Apps	Cloud App Service	Google Migrate for Compute Engine

Common Migration Tools by Cloud Providers:

- AWS (Amazon Web Services):
- AWS Application Migration Service (MGN): Automates lift-and shift migrations.

- AWS Database Migration Service (DMS): Migrates databases to AWS with minimal downtime.
- AWS Snowball: Physical device for offline data transfer.
- Cloud Endure (now part of MGN): Continuous replication with minimal downtime.
- Microsoft Azure:
- Azure Migrate: Unified migration platform for servers, databases, and apps.
- Azure Database Migration Service: Moves on-premises DBs to Azure SQL, Cosmos DB.
- Azure Data Box: Offline data transfer device similar to AWS Snowball.
- Google Cloud Platform (GCP):
- Migrate to Virtual Machines: Migrates VMs to Google Compute Engine.
- Database Migration Service (DMS): Cloud-native database migration tool.
- Transfer Appliance: Offline bulk data transfer device.

Steps to Study a Migration Tool (e.g., AWS MGN):

- **Step 1: Sign in to AWS Console**
  Go to AWS MGN → Enable the service.
- **Step 2: Install Replication Agent**
  On the source server, download and install AWS Replication Agent.
  Configure the agent with your AWS credentials.
- **Step 3: Configure Source Server**
  AWS detects the server and starts real-time block-level replication.
- **Step 4: Launch Test Instances -**
  Launch test instances to verify application functionality.
- **Step 5: Cut Over**
  Launch the migrated server as a live instance and redirect traffic.

Expected Output:

- The migration tool is successfully configured.
- Workload (e.g., a VM or DB) is replicated or transferred to the cloud.
- Migration is verified via test or cut-over instance.
- Dashboard shows success status and minimal downtime.

Oral Questions:

1. What are the "6 R's" of cloud migration?
2. What is AWS Migration Hub used for?
3. What is the difference between AWS DMS and AWS DataSync?
4. When would you use AWS Snowball instead of network transfer?
5. What is "lift and shift" migration?

Conclusion:

Assignment No. 7

Aim: Study and Implement CloudFormation Templates to Provision Requested Resources in an AWS Account

Software/libraries required:

- An active AWS account with access to AWS Management Console.
- Basic knowledge of JSON or YAML syntax.
- Familiarity with AWS services like EC2, S3, IAM, etc.
- Internet connectivity and a computer.
- AWS CloudFormation service enabled in your region.

Theory:

AWS CloudFormation is an Infrastructure as Code (IaC) service that helps you model and set up your AWS resources so that you can spend less time managing infrastructure and more time focusing on your applications. With CloudFormation, you define a template using either YAML or JSON to provision AWS resources such as EC2 instances, S3 buckets, security groups, etc.

The key concepts include:

Template: A configuration file written in JSON or YAML.

Stack: A collection of AWS resources created and managed as a single unit.

Resources: The AWS services you want to create.

Parameters, Mappings, Outputs: Optional elements used to customize templates.

CloudFormation ensures automation, version control, and repeatability of infrastructure deployments.

Steps to Create and Launch a CloudFormation Stack:

Step 1: Log in to AWS Console

- Go to https://aws.amazon.com
- Sign in with your credentials.

Step 2: Open CloudFormation

- Navigate to Services > CloudFormation.
- Click on “Create stack” > “With new resources (standard)”.

Step 3: Upload or Create the Template

- Choose “Upload a template file”.
- Click “Choose file” and upload a pre-written template (e.g., ec2- sample.yaml).
- Alternatively, you can select “Create template in Designer” to visually design one.

Step 4: Configure Stack Details

- Provide a stack name (e.g., MyFirstStack).
- Specify parameters if required (for dynamic templates).

Step 5: Configure Stack Options

- Add optional tags (e.g., Name=DevStack).
- Leave other settings as default for now.

Step 6: Review and Create

- Review the summary and acknowledge that AWS might create IAM resources.
- Click Create stack.

Step 7: Monitor Stack Creation

- You'll be redirected to the Stack events tab.
- Wait until the status changes to "CREATE_COMPLETE".

Step 8: Verify the Created Resources

- Go to EC2 Dashboard (or the relevant service) and verify the resource(s).
- To delete the resources, go to the stack and click "Delete".

Oral Questions:

1. What is AWS CloudFormation, and how does it differ from Terraform?
2. What is a CloudFormation "Stack"?
3. What are the mandatory and optional sections of a CloudFormation template?
4. What happens to resources when a stack is deleted?
5. What template formats does CloudFormation support?

Expected Output:

- CloudFormation template (YAML/JSON) successfully written defining required AWS resources (e.g., EC2, S3, VPC).
- Stack created via CloudFormation console — stack status shows "CREATE_IN_PROGRESS" then "CREATE_COMPLETE."

- All specified resources visible and provisioned correctly under stack's "Resources" tab, matching template definitions.
- Outputs (if defined) displayed correctly under stack's "Outputs" tab (e.g., instance ID, public IP).
- Stack deletion successfully removes all associated resources — status shows "DELETE_COMPLETE."
- Screenshot of stack status and resources list confirming successful provisioning.

Conclusion:

Assignment No. 8

Aim: Implement container management with Kubernetes.

Software/libraries required:

- Basic knowledge of containers and Docker.
- Kubernetes installed via Minikube, Docker Desktop, or using a cloud provider (GKE, EKS, or AKS).
- kubectl command-line tool installed and configured.
- A working container image (e.g., Nginx or custom Docker image).
- Internet connectivity.

Theory:

Kubernetes (also known as K8s) is an open-source container orchestration platform that automates

deployment, scaling, and management of containerized applications.

Key concepts:

- Pod: The smallest deployable unit in Kubernetes, typically wraps one or more containers.
- Node: A physical or virtual machine that runs the containers.
- Cluster: A set of nodes managed by the Kubernetes control plane.
- Deployment: A controller that manages replicas of pods for high availability.
- Service: An abstraction that defines a set of pods and a policy by which to access them (e.g., LoadBalancer, ClusterIP).
- Kubernetes simplifies container management by handling scheduling, updates, networking, and scaling.

Steps to Deploy and Manage a Container Using Kubernetes:

Step 1: Start Minikube (or any K8s cluster)

- Open terminal and start the local Kubernetes cluster: minikube start

Step 2: Create a Deployment

- Create a deployment using a simple container image (e.g., Nginx):

  kubectl create deployment my-nginx --image=nginx

Step 3: Verify Deployment

- Check if the pod is running:

  kubectl get pods

  kubectl get deployments

Step 4: Expose the Deployment as a Service

- To access the Nginx app, expose it via a NodePort:

  kubectl expose deployment my-nginx --type=NodePort --port=80

Step 5: Get the Access URL

- Retrieve the URL to access the application:

  minikube service my-nginx --url

- Open the URL in your browser to see the default Nginx page.

Step 6: Scale the Deployment

- Increase the number of running pods:

  kubectl scale deployment my-nginx --replicas=3

- Check if scaling was successful:
  kubectl get pods

Step 7: Update the Deployment (Optional)

- Roll out a new image version:

  kubectl set image deployment/my-nginx nginx=nginx:1.19

- Watch the rollout status:
  kubectl rollout status deployment/my-nginx

Step 8: Clean Up

- Delete service and deployment after testing:

  kubectl delete service my-nginx

  kubectl delete deployment my-nginx

  minikube stop

Expected Output:

- Kubernetes cluster successfully set up (Minikube/kubeadm) — `kubectl get nodes` shows node status "Ready."
- Pod(s) successfully created and running — `kubectl get pods` shows status "Running."
- Deployment created with specified replicas — `kubectl get deployments` shows desired = current replica count.
- Service exposed (ClusterIP/NodePort) — application accessible via assigned IP/port.
- Scaling command (`kubectl scale`) successfully increases/decreases pod count as verified in `kubectl get pods`.
- Screenshot of terminal output showing nodes, pods, deployments, and services in running state.

Oral Questions:

1. What is container orchestration, and why is Kubernetes needed?
2. What is the difference between a Pod and a Container?
3. What is a Kubernetes Deployment?
4. What does kubectl scale do?
5. What is a Kubernetes Service, and why is it needed?

Conclusion:

Assignment No. 9

Aim: Implement Common Scenarios Using the AWS SDK for Kotlin with IAM

Software/libraries required:

- Kotlin development environment (IntelliJ IDEA or CLI).
- AWS CLI configured (aws configure).
- AWS SDK for Kotlin installed via Gradle or Maven.
- IAM user with programmatic access (Access Key & Secret).
- Basic understanding of Kotlin, AWS IAM, and AWS services like S3 or EC2.

Theory:

The AWS SDK for Kotlin enables developers to interact with AWS services directly from Kotlin applications. It provides Kotlin-native APIs and allows operations such as listing S3 buckets, uploading files, starting/stopping EC2 instances, and more.

IAM (Identity and Access Management) is used to manage access securely by creating users, assigning permissions, and using roles.

IAM + AWS SDK: How It Works

- Create an IAM user with necessary permissions (e.g., S3 read/write).
- Use the user's access key and secret key for authentication.
- The AWS SDK uses these credentials to securely call AWS APIs.

Common Use Cases to Implement:

- Example 1: List All S3 Buckets

```
import aws.sdk.kotlin.services.s3.S3Client
import aws.sdk.kotlin.services.s3.model.ListBucketsRequest
suspend fun listBuckets() {
 S3Client { region = "us-east-1" }.use { s3 ->
   val response = s3.listBuckets(ListBucketsRequest {}) response.buckets?.forEach {
println(it.name) } }
}
```

- Example 2: Upload a File to S3

```
import aws.sdk.kotlin.services.s3.model.PutObjectRequest import java.io.File
import aws.smithy.kotlin.runtime.content.ByteStream
suspend fun uploadFile(bucket: String, key: String, filePath: String) {
 val request = PutObjectRequest {
 this.bucket = bucket
```

```
this.key = key
this.body = ByteStream.fromFile(File(filePath))  }

S3Client { region = "us-east-1" }.use { s3 ->  s3.putObject(request)
println("Uploaded $key to $bucket.")
}
}
```

- Example 3: Describe EC2 Instances

```
import aws.sdk.kotlin.services.ec2.Ec2Client
import
aws.sdk.kotlin.services.ec2.model.DescribeInstancesRequest
suspend fun describeInstances() {
 Ec2Client { region = "us-east-1" }.use { ec2 ->  val response =
ec2.describeInstances(DescribeInstancesRequest {})  response.reservations?.forEach { res ->
 res.instances?.forEach { println("Instance:  ${it.instanceId}") }
 }
 }
}
```

Expected Output:

- The Kotlin program lists all S3 buckets.
- Files are uploaded to the desired S3 bucket programmatically.
- EC2 instances are queried and listed from the application.
- The program interacts securely with AWS using IAM credentials.

Oral Questions:

1. What is the AWS SDK, and why would a developer use it over the Console?
2. What is IAM, and what are its core components (users, groups, roles, policies)?
3. Why is the AWS SDK for Kotlin coroutine-based (suspend functions)?
4. How are AWS credentials securely provided to an SDK application?
5. What is the difference between an IAM user and an IAM role?

Conclusion:

Assignment No. 10

Aim: Demonstrate the Use of Cloud Computing Security Tools (Notable Open Source)

Software/libraries required:

- Linux-based or cloud-hosted environment (local or on AWS/GCP/Azure).
- Internet access and basic command-line skills.
- Basic knowledge of cloud security concepts: IAM, encryption, firewalls.
- Admin privileges on test environment.

Theory:

Cloud security is a critical aspect of cloud computing, ensuring the confidentiality, integrity, and availability of cloud-hosted resources and data. Open-source tools are widely adopted for threat detection, access control, vulnerability scanning, and configuration auditing. These tools help identify misconfigurations, monitor traffic, and secure APIs.

Categories of Open Source Cloud Security Tools:

Category	Tools / Description
Identity & Access	Keycloak, OPA (Open Policy Agent): IAM and policy-based access control
Vulnerability Scanning	OpenVAS, Clair, Trivy: Detects security flaws in OS, containers, etc.
Configuration Auditing	ScoutSuite, Prowler: Audits AWS/Azure/GCP account configurations
Container Security	Falco, Sysdig OSS: Detect suspicious activity in containers/K8s
SIEM / Monitoring	Wazuh, OSSEC: Host and log monitoring, file integrity, threat response

Demonstration: Using Prowler for AWS Security Auditing

Step 1: Install Prowler

git clone https://github.com/prowler-cloud/prowler cd prowler

Step 2: Configure AWS Credentials

aws configure

Step 3: Run Basic Audit
./prowler

Step 4: Run a Specific Check (e.g., S3 Bucket Permissions)
./prowler -c check31

Alternative Tool: Trivy – Container Vulnerability Scanner

Step 1: Install Trivy (Linux)

sudo apt install trivy

Step 2: Scan a Docker image

trivy image nginx:latest

Expected Output:

- Tools like Prowler generate HTML/JSON/terminal reports showing misconfigurations and compliance scores.
- Trivy lists CVEs found in container images.
- Insight into cloud security posture and weaknesses is gained.

Oral Questions:

1. What is Cloud Security Posture Management (CSPM)?
2. What does ScoutSuite/Prowler check for in a cloud account?
3. Name two common cloud misconfigurations that lead to security breaches.
4. What is the "shared responsibility model" in cloud security?
5. What is Cloud Custodian used for?

Conclusion: